

AI Study Companion

Architecture Documentation — how the system is built, and why it is built that way

Product	AI-Powered Learning & Growth Workspace (PRD v3.0, Candidate Challenge Edition)
Live deployment	https://ai-study-companion-three-inky.vercel.app/
Stack	Next.js 14 + TypeScript · Supabase (Postgres + pgvector + Auth + Storage) · Inngest · Mercury mercury-2.5 · Gemini gemini-embedding-2 (768-d)
Version / Date	v1.0 — 18 September 2026 · Prototype scope (3–4 days)
Test status	314 tests across 26 files passing · 18 eval fixtures · production build clean

Contents

1. Product overview & the closed learning loop
2. Technology choices & justification
3. High-level architecture
4. Frontend architecture
5. API & service layer
6. Database & data isolation
7. RAG & grounded Tutor (citations, unsupported handling, injection defense)
8. Adaptive quiz, assessment, mastery, growth & recommendations
9. Persistent learning context
10. Events & intelligent background workflows
11. AI engineering, observability & evaluation
12. Reliability, security & performance
13. Deployment & configuration
14. Testing summary · Trade-offs & limitations · Future work

1. Product overview & the closed learning loop

AI Study Companion is a persistent, contextual, measurable learning workspace — deliberately *not* a PDF chatbot. A learner organises study into **Spaces** (broad areas) → **Projects** (focused goals), uploads **Materials** (PDFs + images), learns with a **grounded AI Tutor**, is tested by an **adaptive Quiz** (MCQ + open-ended), and watches **Concept Mastery** evolve into **Growth Analysis** and actionable **Recommendations**. An **Admin Dashboard** gives platform-level visibility into users, learning, AI usage, evaluation and system health.

The defining architectural idea is a **closed evidence loop**: every AI feature either *produces* learning evidence or *acts on* it. Nothing stands alone.

Create Space → Create Project → Upload Material → Process & Index → Learn with Tutor (grounded + cited) → Adaptive Quiz → Evaluate → Update Mastery → Analyse Growth → Recommend Next Action → Continue Learning

🔄 each step emits events; each event can trigger the next workflow

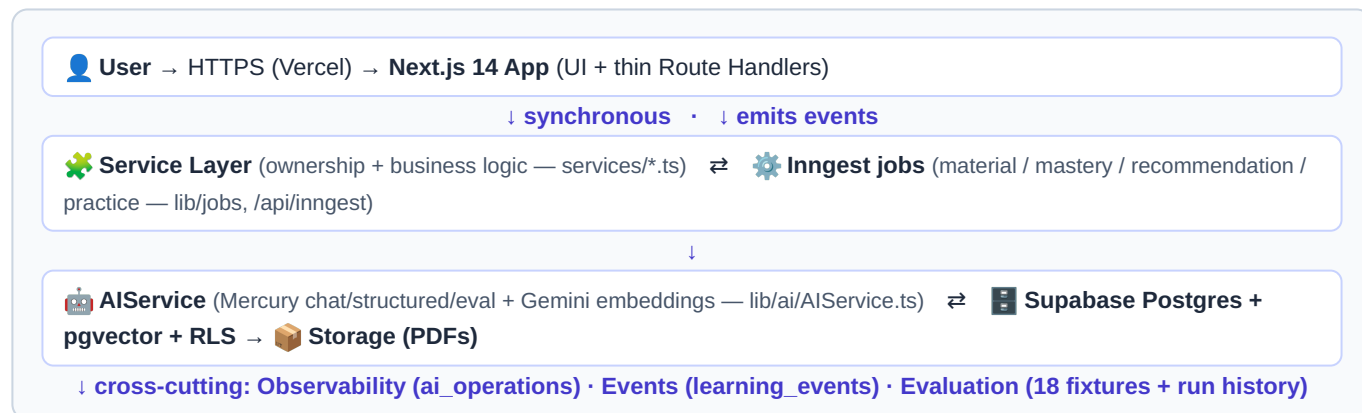
Design principles: Context-first (never leak across Projects) · Evidence over guessing (say “insufficient evidence” rather than hallucinate) · Persistent-but-relevant context (bounded, not full history) · Asynchronous by design · Observable AI · Controlled AI ↔ app interaction (validated tools, never raw DB access).

2. Technology choices & justification

Layer	Choice (actual, shipped)	Why — engineering judgement
App framework	Next.js 14 App Router + TypeScript	Full-stack in one repo, strongest stack for speed; thin route handlers + server components; one-command Vercel deploy.
UI	React + Tailwind (stone/sky theme, shared Sidebar)	No custom design system; consistent project rail = the learning loop made visible.
Database	Supabase Postgres (single source of truth)	Relational domain (User → Space → Project → ...) fits Postgres; one project for DB + Auth + Storage.
Vector search	pgvector in the same Postgres, chunks.embedding VECTOR(768)	No second vector DB to operate; relational + vector predicates in one query; match_chunks RPC scoped by project_id.
Auth	Supabase Auth (JWT, sessions, RLS)	No custom auth; middleware guards app routes; service-layer ownership checks + RLS defence-in-depth.
File storage	Supabase Storage, private materials bucket	Same project as DB/Auth; paths scoped user/project/file, sanitised filenames.
Background jobs	Inngest (Cloud) + inline fallback	Event-driven with retries, zero infra; maxDuration=60 + direct-processing fallback so Vercel serverless never leaves materials stuck in QUEUED.
Chat / structured / eval	Mercury (Inception Labs), model mercury-2.5 via lib/ai/AIService.ts	OpenAI-compatible + JSON mode; reasoning-model shaping (token headroom, temp clamp, reasoning_effort=low for structured tasks); server-side schema validation on every output.
Embeddings	Google Gemini gemini-embedding-2, 768 dims	Free-tier eligible; Matryoshka-native 768 preserves existing VECTOR(768) column (migration 012); same model+config for documents and queries; key server-only, never client.
Deployment	Vercel (app) + Supabase (data) + Inngest (jobs)	Zero-ops for a solo 3–4 day prototype; env-separated, secrets never committed.

Deliberately not built: microservices, Kubernetes, a dedicated vector DB, custom auth/queues, multi-provider routing, distributed tracing. Each would cost a day with zero observable payoff at prototype scale. See §14.

3. High-level architecture



Golden rule: every request — sync or background — passes *Authentication* → *Authorization* → *Ownership validation* → *Business logic* → *Database (RLS as second layer)* before any data is touched.

4. Frontend architecture

App Router, server components by default; client components only where interactivity demands (Tutor chat, quiz taking, forms). One shared components/Sidebar.tsx shell (workspace / project / admin, mobile drawer).

```
app/ / /login /signup /dashboard (Home: Continue · Recent · Progress · Attention · Next Action)
/spaces /spaces/[spaceId] /profile
/projects/[projectId] (hub) + materials · tutor · quiz · practice · flashcards
                        · concepts · mastery · growth · analytics · recommendations
/admin/ dashboard · users · users/[userId] · spaces · projects · activity
      engagement · learning · ai-usage · ai-evaluation · jobs · health
```

Inside a Project the rail **Materials · Tutor · Quiz · Mastery · Growth · Analytics · Recommendations** (plus Practice, Flashcards, Concepts) is always visible — the loop made navigable. Every async view implements **Loading** **Processing** **Ready** **Failed+Retry** **Empty**.

5. API & service layer

Route handlers stay **thin**: they resolve the session, call a service, map errors (401/404/429). All logic lives in services/; all AI lives in ai/ behind AIService. A handler never touches the DB directly.

Area	Key files	Responsibility
Learning CRUD	project, material, chunk, concept, flashcard services	Spaces/Projects, upload, chunks, concepts, decks — all ownership-scoped.
AI orchestration	tutor, quiz, recommendation, practice, misconception, knowledge-graph services	RAG orchestration, adaptive selection, grading, recommendations.
Learning science	mastery.service.ts, growth.service.ts	Deterministic 0.7/0.3 updates; ± 5 trend classification.
Insight	analytics, dashboard, admin, evaluation services	Project/global aggregations; Home “what next”; admin drill-downs; run-over-run eval comparison.
Prompts & schemas	ai/tutor, quiz, assessment, concepts, recommendation, flashcards, practice.ts	System prompts + JSON schemas + server-side validators per feature.
Cross-cutting	lib/auth, lib/db, lib/rag, lib/storage, lib/jobs, lib/security	Sessions, typed queries, retrieval, uploads, Inngest registration, rate limits.

6. Database & data isolation

One Postgres holds relational + vector data. Simplified schema (uuid PKs, timestampz, CASCADE to parents):

```
spaces (id, user_id, name, description)
projects (id, space_id, user_id, name, description, learning_goal)
materials (id, project_id, user_id, filename, storage_path, file_hash, file_size,
           status QUEUED|PROCESSING|READY|FAILED, page_count, processing_error)
chunks (id, material_id, project_id, content, page_number, chunk_index,
        embedding VECTOR(768), metadata JSONB)
concepts (id, project_id, name, description, source_material_id)
concept_mastery (id, project_id, concept_id, user_id, mastery_score, evidence JSONB)
mastery_history (id, concept_id, user_id, previous_score, new_score, reason) -- growth input
conversations (id, project_id, user_id, summary, summary_updated_at, pinned) / messages (... , citations JSONB)
quizzes / questions (MCQ|OPEN_ENDED, difficulty, options, correct_answer, explanation)
answers (response, is_correct, score, evaluation JSONB)
practice_assignments / practice_items / misconceptions (repeated-mistake tracking)
recommendations (title, action_items JSONB, ACTIVE|COMPLETED|DISMISSED)
learning_events (user/space/project, event_type, entity, metadata) -- activity + analytics + admin
ai_operations (feature, model, request_id, latency_ms, success, tokens, cost, error)
```

Isolation (defence in depth): service functions scope every lookup — WHERE id=\$1 AND user_id=\$2, never bare id; misses return **404** (no existence leak); RLS user_id = auth.uid() on all user tables as an independent second layer; background jobs carry user_id/project_id in the event payload; retrieval filters WHERE project_id always. Migrations 001–014 apply in order (npm run migrate or SQL editor); indexes on all FKs, learning_events(project_id, created_at), and hnsw/ivfflat on chunks.embedding.

7. RAG & grounded Tutor

7.1 Indexing pipeline (async)

Upload (PDF/png/jpg/webp, 10 MB, non-empty, SHA-256 dedupe per project) → Storage → materials QUEUED → Inngest job (inline fallback)

↓

Claim (atomic QUEUED/FAILED → PROCESSING) → extract text w/ page numbers (pdf-parse; OCR for scans) → reject <20 chars → chunk (~2400 chars / 320 overlap) → Gemini embed (batches of 20, dim-gated) → insert (batches of 50, stale deleted) → concepts (≤8, non-fatal) → READY / FAILED+error

7.2 Tutor query path

Authenticate → validate project ownership → dedupe (30 s) → persist user turn → embed query (Gemini, 10-min cache) → match_chunks scoped by project_id (over-fetch ≤100, JS-cosine fallback) → filter RELEVANCE_THRESHOLD=0.25, balance across materials, top-K 5

↓

Insufficient evidence? → skip LLM, persist fixed “couldn't find enough evidence...” response · **Else** → compose (chunks + 6-msg window + rolling summary + learning/assessment context) → generateStructured (Mercury, low temp) → validate schema + filter citations to real chunk IDs → persist + log

Citations: every grounded answer returns {answer, confidence, grounded, citations[{materialId, materialName, page, chunkId}], followUp}; UI shows “Source: Name — Page N” with excerpts via ownership-checked /api/chunks/[id].

Prompt-injection defense: retrieved text is *data* inside a delimited <retrieved_evidence> block marked untrusted; system prompts forbid following instructions inside it. Verified by fixture TUTOR-05 (malicious PDF).

8. Adaptive quiz, assessment, mastery, growth & recommendations

Capability	How it works (shipped)
Adaptive selection	Weighted score per concept: low mastery + recent mistakes + decline/improve deltas + recency/frequency (anti-repeat); difficulty + MCQ-vs-open-ended picked by thresholds — never naive wrong → easy.
Generation	Mercury structured output {concept_id, type, difficulty, question, options[4], correct_answer, explanation}; server validation (exactly 4 options, correct ∈ options); retry once at lower temp; 2-min duplicate-quiz guard; answer gating (correct/explanation hidden until graded).
Grading	MCQ deterministic string compare (no LLM). Open-ended: LLM → {score 0–100, understanding, strengths, missingConcepts, reasoningQuality, feedback}; score ≥ 60 ⇒ correct; failure raises, never writes a guess; per-question idempotency (re-POST returns existing).
Mastery (deterministic)	$new = prev \times 0.7 + evidence \times 0.3$ (MCQ 0/100, open-ended as-is, averaged per concept), clamped/rounded; upsert + mastery_history row + MASTERY_UPDATED. The LLM never sets a score.
Growth	Last two history points per concept: >+5 IMPROVING, <−5 REQUIRES_ATTENTION, else STABLE.
Recommendations	After mastery updates: weak concepts (score<60 or REQUIRES_ATTENTION) + mistakes + goal + activity → Mercury → {title, action_items[2–5]} naming real concepts/materials; skipped when nothing is weak; COMPLETED/DISMISSED lifecycle.

Differentiators on top of the baseline: Flashcards (weak-concept decks), Deep Practice (sections + MCQ), Misconception tracking (repeated-mistake workflow), Knowledge Graph, Sub-concept expansion, conversation pinning.

9. Persistent learning context

Only *useful* context is stored or sent — never full transcripts:

Request → Project Knowledge (RAG chunks = evidence) + Conversation (last 6 msgs, always) + Rolling Summary (older turns, ≤1200 chars, refreshed ~every 6 msgs past 8) + Learning Context (goal, weak concepts, repeated mistakes) + Assessment Context (recent quiz/answers) → Compose → LLM

Summary is *context-only* (never evidence, never instructions) and refresh is best-effort — it never blocks the answer. This is what separates a companion from a chatbot.

10. Events & intelligent background workflows

One `learning_events` table powers the activity feed, analytics, recommendations and admin. Catalog includes `SPACE/CREATED`, `PROJECT_CREATED`, `MATERIAL_UPLOADED/PROCESSING_STARTED/READY/FAILED`, `TUTOR_MESSAGE_SENT/RESPONSE_GENERATED`, `QUIZ_STARTED`, `QUESTION_ANSWERED`, `QUIZ_COMPLETED`, `MASTERY_UPDATED`, `RECOMMENDATION_GENERATED/COMPLETED` (+ practice/misconception events).

Material: Upload → Process → Chunk → Embed → Concepts → READY

Learning: Quiz Completed → Evaluate → Update Mastery → Detect Weakness → Insight → Recommend

Repeated-mistake: Pattern (misconception bump/merge) → Update Context → Targeted Recommendation

Idempotency & recovery: atomic material claim; file-hash dedupe (duplicate → reuse, no second job); 2-min quiz-generation guard; per-question answer idempotency incl. concurrent-race handling; unique constraint where duplication harms; FAILED materials retry inline (heals even if Inngest delivery failed); user never needs the browser open.

11. AI engineering, observability & evaluation

AI is treated as an engineered system: one `AIService` abstraction (chat / structured / embeddings / evaluation / doc-understanding), JSON-mode + per-feature validators, 90 s timeouts, typed fallbacks.

Observability: every call writes one `ai_operations` row — `feature`, `model`, `request_id`, `latency_ms`, `success`, `tokens`, `estimated_cost`, `error` — for `TUTOR`, `CONVERSATION_SUMMARY`, `EMBEDDING`, `QUIZ_GENERATION`, `OPEN_ENDED_EVALUATION`, `CONCEPT_EXTRACTION`, `RECOMMENDATION`, `FLASHCARD_GENERATION`, `SUBCONCEPT_GENERATION` (+ practice). Surfaced in `/admin/ai-usage` and `project/global analytics` — answering *why slow? which model? why poor retrieval? which workflow failed? what cost?*

Evaluation: 18 curated fixtures (`tests/eval/fixtures.ts`) — Tutor (grounded / unsupported / multi-concept / citation / injection), Retrieval (query → expected source), Assessment (missing-concept detection), Recommendations (specificity rubric). `npm run eval` stamps `runId/suiteVersion`, archives per-run history, and `/admin/ai-evaluation` shows run-over-run IMPROVED REGRESSED UNCHANGED BASELINE. Regression-aware by design: prompt/model/retrieval changes are diffed, not silently shipped.

12. Reliability, security & performance

Risk	Handling (shipped)
AI timeout / provider failure	90 s timeout; retry once w/ backoff; typed user-safe error; UI never stuck on Loading.
Invalid AI output	Schema reject + log (success=false) + safe fallback; open-ended grading raises instead of persisting guesses.
PDF / retrieval failure	FAILED + processing_error + Retry; empty retrieval = insufficient-evidence path, not an error.
Auth / ownership	Routes: 401 unauthenticated, 404 not-owned (no leak); pages redirect to /login; writes scoped by user_id.
Cost spikes	In-memory per-user fixed windows (tutor 20/min, quiz-gen 5/min, quiz-submit 60/min, flashcards 5/min, subconcepts 10/min) → 429 + Retry-After; embeddings cached 10 min; no re-embed of READY materials.
Duplicates on retry	Hash dedupe, atomic claims, idempotent answers, generation guards — retries never duplicate state.

Performance (prototype-appropriate): pgvector top-K + threshold (no full scans in app code); balanced-per-material retrieval; paginated analytics/admin; async everything long-running; no unnecessary AI calls (summary best-effort, embeddings only for real work). Streaming Tutor and response caching are documented future work, not regressions.

13. Deployment & configuration

Vercel (Next.js app, /api/ingest synced to Inngest Cloud) + Supabase (Postgres/pgvector, Auth, private materials bucket) + Inngest (jobs) + Mercury + Gemini (AI)

Setup: import repo → set env (see .env.example: DATABASE_URL, NEXT_PUBLIC_SUPABASE_URL/ANON_KEY, SUPABASE_SERVICE_ROLE_KEY (server-only), MERCURY_API_KEY/BASE_URL/CHAT_MODEL, GEMINI_API_KEY/MODEL/DIM, INNGEST_EVENT/SIGNING_KEY (+DEV=1 local), ADMIN_EMAILS/ADMIN_USER_IDS) → run migrations 001→014 → ensure bucket → sync Inngest → smoke-test the full loop live. Secrets are env-only, gitignored, never committed; service-role key never reaches the browser.

14. Testing · trade-offs · future

Testing (meaningful, not exhaustive): 314 unit/integration tests across 26 files — mastery math, ownership/IDOR, tutor insufficient path + citation filtering + summary continuity, quiz gating, rate-limit windows, 401/404 mapping, admin health, eval run-tracking, Gemini formatting + mocked errors, migration ordering, scanned-PDF OCR, dedupe hashing. Plus 18 eval fixtures with live + offline scoring. npm test · npm run eval · npm run build · npm run lint · npm run typecheck all green.

Trade-off (chosen)	Why	Production path
Mastery = 0.7/0.3 average; growth = ± 5	Explainable, testable in hours	Difficulty-weighted + forgetting curves + confidence bands
Admin = allow-list, no RBAC	Zero-schema authz for prototype	<code>profiles.is_admin</code> + policies + audit log
Eval = curated fixtures + file history	Real signal without eval-platform build	LLM-as-judge + dataset versioning + CI gates
Rate limit = single-instance memory	Cost guard in minutes	Shared store (Redis) for multi-instance
No streaming / no cache	Correctness first on the clock	Token streaming + embedding/response cache

Known limitations (honest): token/cost columns exist but provider usage is not yet threaded through (calls/latency/errors visible, cost estimated); quiz-completed fallback leans on Inngest delivery (inline heal exists for materials); injection delimiters cover Tutor/practice/concepts, remaining prompts queued for the same treatment; uploads have file-level (not per-user quota) limits.

Future (schema already supports): Learning Path Engine (order concepts+mastery) · Mistake Intelligence (cluster `answers.evaluation`) · Concept Graph edges · Socratic Tutor mode · Spaced Repetition (`mastery_history.created_at`) · Evidence-aware UI (`mastery.evidence`) · cross-project AI Coach.

Success criterion — met: a user completes Space → Project → Material → Knowledge → Tutor (+citation) → Unsupported handling → Adaptive Quiz → Assessment → Mastery → Growth → Analytics → Recommendation → Continue Learning without losing context, while an admin simultaneously inspects users, projects, activity, analytics, AI usage, evaluation and health. Live at the URL on the cover.